مستند آموزشی و راهنمای یادگیری

DESIGN SYSTEM

گردآورنده: مرضیه نجف زاده

فهرست مطالب

- مقدمه و تعریف سیستم طراحی
- اصول پایه و مبانی طراحی
- توکن‌های طراحی و لایه‌های آن
- کامپوننت‌ها، حالت‌ها، گونه‌ها و طراحی API
- ترکیب کامپوننت‌ها، طراحی اتمی و الگوها
- چیدمان، طراحی واکنش‌گرا و آیکون‌نگاری
- دسترس‌پذیری و حرکت
- مستندسازی و Storybook
- حاکمیت، نسخه‌بندی و آزمون
- فرایند ساخت سیستم طراحی و ممیزی رابط کاربری
- مقایسه سیستم طراحی، کتابخانه رابط کاربری و راهنمای سبک
- مسیر پیشنهادی یادگیری و جمع‌بندی

۱. مقدمه

با بزرگ‌تر شدن محصولات نرم‌افزاری، حفظ هماهنگی در رابط کاربری به یکی از چالش‌های اصلی تیم‌های طراحی و توسعه تبدیل می‌شود. در یک محصول کوچک ممکن است طراحی چند دکمه، فرم و صفحه بدون ساختار مشخص مشکل جدی ایجاد نکند؛ اما با افزایش تعداد صفحات، تیم‌ها و قابلیت‌ها، تصمیم‌های پراکنده به سرعت به ناهماهنگی تبدیل می‌شوند.

Design System راهی برای تبدیل تصمیم‌های تکرارشونده طراحی به قواعدی مشخص، قابل استفاده مجدد و قابل نگهداری است. هدف آن فقط زیباتر کردن رابط نیست؛ بلکه کاهش تصمیم‌گیری‌های تکراری، افزایش کیفیت، سرعت توسعه و ایجاد زبان مشترک میان طراحی و توسعه است.

- تفاوت در رنگ‌ها و تایپوگرافی بین صفحات
- وجود چند نسخه متفاوت از یک کامپوننت مشابه
- افزایش کد و تصمیم‌های تکراری
- دشواری نگهداری و اعمال تغییرات سراسری
- کاهش سرعت توسعه و افزایش خطای انسانی
- ناهماهنگی میان تیم طراحی، توسعه و محصول

۲. تعریف سیستم طراحی

Design System مجموعه‌ای ساختاریافته از اصول، استانداردها، Design Tokenها، کامپوننت‌ها، الگوهای تعامل، مستندات و فرایندهای مدیریتی است که برای ساخت تجربه‌ای هماهنگ و قابل توسعه استفاده می‌شود.

در عمل، Design System به پرسش‌هایی مانند «رنگ اصلی چیست؟»، «فاصله استاندارد چقدر است؟»، «Button چه حالت‌هایی دارد؟»، «خطا چگونه نمایش داده می‌شود؟» و «چه زمانی باید یک Component جدید ساخته شود؟» پاسخ می‌دهد.

```
Design System
│
├── Foundations
├── Design Tokens
├── Components
├── Patterns
├── Layout System
├── Accessibility
├── Documentation
└── Governance
```

۳. اصول پایه

Foundations پایه‌های بصری و رفتاری سیستم هستند. بهتر است پیش از ساخت Componentها، این تصمیم‌های پایه مشخص شوند تا اجزای مختلف محصول بر مبنای قواعد مشترک شکل بگیرند.

```
Color
Typography
Spacing
Grid
Border Radius
Elevation
Icons
Breakpoints
Motion
```

۴. سیستم رنگ

رنگ‌ها در Design System نباید صرفاً مجموعه‌ای از Hex Codeها باشند. هر رنگ باید نقش و معنای مشخص داشته باشد. نام‌گذاری معنایی باعث می‌شود کاربرد رنگ‌ها قابل پیش‌بینی باشد و تغییر برند یا Theme آسان‌تر انجام شود.

```
Primary
Secondary
Success
Error
Warning
Info
Background
Surface
Text
Border
```

```
const colors = {
  primary: "#2563EB",
  secondary: "#64748B",
  success: "#16A34A",
  error: "#DC2626",
  warning: "#D97706",
  background: "#F8FAFC",
};
```

۵. تایپوگرافی

Typography فقط انتخاب فونت نیست. اندازه، وزن، ارتفاع خط و سلسله‌مراتب متن باید استاندارد باشند تا خوانایی و ریتم بصری در تمام رابط حفظ شود.

- Font Family
- Font Size
- Font Weight
- Line Height
- Letter Spacing
- Text Hierarchy

```
Display
Heading 1
Heading 2
Heading 3
Body
Caption
Label
```

```
const typography = {
  h1: { fontSize: "32px", fontWeight: 700, lineHeight: 1.3 },
  h2: { fontSize: "24px", fontWeight: 600, lineHeight: 1.4 },
  body: { fontSize: "14px", fontWeight: 400, lineHeight: 1.7 },
};
```

۶. سیستم فاصله‌گذاری

Spacing System فاصله میان عناصر را به مجموعه‌ای محدود و قابل پیش‌بینی تبدیل می‌کند. استفاده از مقیاس ثابت، نظم بصری ایجاد می‌کند و از شکل‌گیری مقادیر تصادفی جلوگیری می‌کند.

```
4px  8px  12px  16px  24px  32px  40px  48px
```

> اصل مهم
> قرار نیست همه فاصله‌ها دقیقاً یک اندازه باشند؛ هدف این است که فاصله‌ها از یک مقیاس مشخص انتخاب شوند و تصمیم‌های طراحی قابل تکرار باشند.

۷. گوشه‌های گرد و ارتفاع بصری

Radius و Shadow نیز بخشی از زبان بصری محصول هستند. تعریف چند سطح محدود کمک می‌کند Cardها، Modalها، Inputها و سایر سطوح، ظاهر هماهنگ داشته باشند.

```
const radius = {
  sm: "4px",
  md: "8px",
  lg: "12px",
  xl: "16px",
};
```

```
const shadows = {
  sm: "0 1px 2px rgba(0,0,0,0.06)",
  md: "0 4px 12px rgba(0,0,0,0.08)",
  lg: "0 10px 25px rgba(0,0,0,0.12)",
};
```

۸. توکن طراحی چیست؟

Design Token یک مقدار طراحی نام‌گذاری‌شده است. به جای استفاده مستقیم از مقادیر خام، از نامی استفاده می‌کنیم که هدف یا نقش آن مقدار را بیان کند. Tokenها پلی میان طراحی و پیاده‌سازی هستند.

```
Raw value:      #2563EB
Semantic token: color.primary
```

```
const tokens = {
  color: { primary: "#2563EB", success: "#16A34A", error: "#DC2626" },
  spacing: { xs: "4px", sm: "8px", md: "16px", lg: "24px" },
  radius: { sm: "4px", md: "8px", lg: "12px" },
};
```

۹. لایه‌های توکن طراحی

Primitive Tokens

مقادیر پایه و خام مانند رنگ‌ها، فاصله‌ها یا اندازه‌ها هستند.

```
blue-500
gray-200
spacing-4
radius-8
```

Semantic Tokens

نام آن‌ها بر اساس کاربرد تعریف می‌شود و به جای ظاهر، معنا را منتقل می‌کنند.

```
color-primary
color-error
text-primary
background-surface
border-default
```

Component Tokens

برای کنترل جزئیات یک Component خاص استفاده می‌شوند و معمولاً به Semantic Tokenها متصل هستند.

```
button-primary-background
button-primary-text
input-border-color
card-border-radius
```

۱۰. کامپوننت چیست؟

Component یک واحد قابل استفاده مجدد از رابط کاربری است. یک Component حرفه‌ای فقط ظاهر ندارد؛ API، رفتار، حالت‌ها، محدودیت‌ها و مستندات آن نیز مشخص هستند.

```
Button
Input
Select
Checkbox
Radio Button
Modal
Tabs
Card
Tooltip
Table
Pagination
```

- قابل استفاده مجدد باشد
- API روشن و قابل پیش‌بینی داشته باشد
- رفتار آن در شرایط مختلف مشخص باشد
- Stateها و Variantهای آن تعریف شده باشند
- مستند و قابل تست باشد

۱۱. حالت‌ها و گونه‌ها

State وضعیت لحظه‌ای Component را توصیف می‌کند؛ Variant نسخه طراحی یا کاربردی آن Component است. تفکیک این دو مفهوم باعث طراحی API شفاف‌تر می‌شود.

مفهوم	نمونه‌ها	تعریف
State	Default، Hover، Focus، Active، Disabled، Loading	وضعیت فعلی یا تعاملی Component
Variant	Primary، Secondary، Outlined، Text، Danger	نسخه هدفمند یا سبک بصری Component

۱۲. طراحی API کامپوننت

API مشخص می‌کند توسعه‌دهنده چگونه با Component تعامل دارد. نام Propها باید کوتاه، قابل فهم و هم‌راستا با واژگان عمومی رابط کاربری باشند. API بیش از حد آزاد، عملاً استاندارد را از بین می‌برد.

```
<Button
  variant="primary"
  size="medium"
  disabled={false}
>
  Submit
</Button>
```

```
variant
size
disabled
loading
icon
onClick
```

۱۳. ترکیب کامپوننت‌ها

Composition یعنی ساخت اجزای بزرگ‌تر از ترکیب Componentهای کوچک و مستقل. این رویکرد انعطاف‌پذیری را افزایش می‌دهد و از ساخت Componentهای بسیار سنگین با تعداد زیادی Prop جلوگیری می‌کند.

```
Form
│
├── Label
├── Input
├── Helper Text
├── Error Message
└── Button
```

۱۴. طراحی اتمی

Atomic Design یکی از مدل‌های شناخته‌شده برای دسته‌بندی سطح پیچیدگی عناصر رابط است. این مدل الزام معماری نیست، اما برای تفکر درباره ترکیب Componentها مفید است.

سطح	نمونه	توضیح
Atoms	Button، Input، Icon، Label	کوچک‌ترین واحدهای رابط
Molecules	Search Field، Form Field	ترکیب چند Atom
Organisms	Header، Sidebar، Data Table	بخش‌های بزرگ‌تر و مستقل صفحه
Templates	Dashboard Layout، Form Layout	ساختار کلی صفحه بدون داده نهایی
Pages	صفحه واقعی محصول	Template همراه داده و محتوای واقعی

۱۵. الگو چیست؟

Pattern یک راه‌حل استاندارد برای سناریوهای تکرارشونده است. Component می‌گوید «چه چیزی داریم»، اما Pattern می‌گوید «چگونه از چند جزء برای حل یک مسئله استفاده کنیم».

```
Search Pattern
Delete Confirmation Pattern
Form Validation Pattern
Empty State Pattern
Error Handling Pattern
Filtering Pattern
```

```
User clicks Delete
      ↓
Confirmation Modal
      ↓
User confirms
      ↓
Delete Request
      ↓
Success Feedback
```

۱۶. سیستم چیدمان

Layout System قواعد چیدمان را استاندارد می‌کند. این قواعد شامل Grid، عرض Container، فاصله کناره‌های صفحه، تعداد ستون‌ها، Alignment و رفتار Responsive هستند.

- Grid System
- Container Width
- Page Padding
- Column Rules
- Alignment
- Responsive Behavior

```
12 Column Grid
```

۱۷. طراحی واکنش‌گرا

Responsive Design در Design System فقط تعیین چند Breakpoint نیست. باید مشخص شود تایپوگرافی، Grid، Sidebar، Modal، Table، Navigation و Spacing در اندازه‌های مختلف چگونه تغییر می‌کنند.

```
xs  sm  md  lg  xl
Mobile → Tablet → Laptop → Desktop → Large Desktop
```

۱۸. دسترس‌پذیری

Accessibility باید از ابتدا بخشی از Design System باشد. در این صورت، هر محصولی که از Componentهای سیستم استفاده می‌کند بخشی از الزامات دسترس‌پذیری را به صورت پیش‌فرض دریافت می‌کند.

- Color Contrast
- Keyboard Navigation
- Visible Focus State
- Screen Reader Support
- ARIA Attributes
- Semantic HTML

• Touch Target Size

```
button:focus-visible {
  outline: 2px solid currentColor;
  outline-offset: 2px;
}
```

۱۹. آیکون‌نگاری

آیکون‌ها نیز باید تحت قواعد مشترک باشند. استفاده از چند سبک آیکون متفاوت یا اندازه‌های نامنظم باعث کاهش یکپارچگی بصری می‌شود.

• سبک بصری
• اندازه‌های استاندارد
• Stroke Width
• Filled یا Outlined بودن
• رنگ
• فاصله با متن

```
16px  20px  24px  32px
```

۲۰. حرکت و انیمیشن

Motion بخشی از تجربه کاربری است و باید هدفمند باشد. مدت زمان Animation، منحنی حرکت و رفتار Transition بهتر است به صورت Token تعریف شوند تا حس حرکت در محصول یکپارچه باشد.

```
const motion = {
  fast: "120ms",
  normal: "200ms",
  slow: "300ms",
};
```

```
Ease In
Ease Out
Ease In Out
```

۲۱. مستندسازی

حتی بهترین Component بدون Documentation به مرور به منبع ابهام تبدیل می‌شود. مستند خوب باید علاوه بر API، منطق استفاده صحیح را نیز توضیح دهد.

• هدف Component
• زمان مناسب استفاده
• زمانی که نباید استفاده شود
• Props و API
• Variants و States
• مثال‌های صحیح

• Accessibility Notes
• Design Guidelines

۲۲. Storybook

Storybook یکی از ابزارهای رایج برای توسعه و مستندسازی Componentها در محیطی مستقل از محصول است. هر Story یک حالت مشخص از Component را نمایش می‌دهد و می‌تواند به عنوان مرجع زنده سیستم استفاده شود.

```
Default
Primary
Secondary
Disabled
Loading
With Icon
```

```
export default {
  title: "Components/Button",
  component: Button,
};

export const Primary = {
  args: { variant: "primary", children: "Submit" },
};
```

- Documentation
- Interaction Testing
- Accessibility Testing
- Visual Regression Testing

۲۳. حاکمیت سیستم طراحی

Governance مجموعه قواعدی است که مشخص می‌کند Design System چگونه رشد و مدیریت شود. بدون Governance، حتی یک کتابخانه مرتب نیز ممکن است به مرور به مجموعه‌ای نامنسجم از تصمیم‌ها تبدیل شود.

- چه کسی می‌تواند Component جدید پیشنهاد یا اضافه کند؟
- چه زمانی Component جدید واقعاً لازم است؟
- Breaking Change چگونه بررسی و اعلام می‌شود؟
- چه کسی تغییرات Tokenها را تأیید می‌کند؟
- Deprecated Componentها چگونه مدیریت می‌شوند؟
- نسخه‌ها و Releaseها چگونه منتشر می‌شوند؟

۲۴. نسخه‌بندی

اگر Design System به صورت Package مصرف شود، Versioning برای مدیریت تغییرات ضروری است. یکی از روش‌های رایج Semantic Versioning است.

```
MAJOR.MINOR.PATCH

2.4.1
```

کاربرد	نمونه	نوع تغییر
رفع Bug بدون تغییر API	2.4.2 → 2.4.1	PATCH
قابلیت جدید بدون Breaking Change	2.5.0 → 2.4.2	MINOR
تغییر ناسازگار با نسخه قبلی	3.0.0 → 2.5.0	MAJOR

۲۵. سیستم طراحی، کتابخانه رابط کاربری و راهنمای سبک

این سه اصطلاح به هم مرتبط هستند، اما یکسان نیستند. UI Library بیشتر روی اجزای قابل استفاده مجدد تمرکز دارد؛ Style Guide بر قواعد بصری متمرکز است؛ Design System علاوه بر این دو، الگوها، مستندات، دسترس‌پذیری، Governance و فرایند تکامل سیستم را نیز در بر می‌گیرد.

مفهوم	تمرکز اصلی	نمونه محتوا
Style Guide	هویت و قواعد بصری	Color، Typography، Logo، Icon
UI Library	کامپوننت‌های قابل استفاده مجدد	Button، Input، Modal، Table
Design System	سیستم جامع تصمیم‌های طراحی و توسعه	Foundations، Tokens، Components، Patterns، Docs، Governance

۲۶. جلوگیری از انباشت بی‌قاعده کامپوننت‌ها

یکی از خطاهای رایج این است که هر Component وارد Design System شود. Componentهایی که فقط برای یک سناریوی خاص، دامنه تجاری خاص یا صفحه خاص ساخته شده‌اند، الزاماً جزء Design System نیستند.

یک Component مناسب برای Design System معمولاً قابلیت استفاده در چند Context را دارد، وابستگی مستقیمی به Business Logic ندارد، API عمومی و رفتار نسبتاً پایدار دارد.

۲۷. تعادل در عمومی‌سازی کامپوننت‌ها

در نقطه مقابل، Component نباید آن‌قدر Generic شود که مصرف‌کننده بتواند تقریباً همه تصمیم‌های ظاهری را از طریق Propها تغییر دهد. در این حالت استاندارد از بین می‌رود و Component به یک Wrapper ضعیف تبدیل می‌شود.

```
<Button
  backgroundColor
  hoverColor
  borderColor
  borderWidth
  radius
  fontSize
  fontWeight
  shadow
  padding
/>
```

بهتر است آزادی مصرف‌کننده از طریق Variant و Sizeهای محدود، مشخص و مستند کنترل شود.

```
<Button variant="primary" size="medium">
  Submit
</Button>
```

۲۸. فرایند ساخت سیستم طراحی

ساخت Design System بهتر است مرحله‌ای و مبتنی بر نیاز واقعی انجام شود. ساخت همه اجزا از ابتدا معمولاً هزینه زیادی ایجاد می‌کند و ممکن است به تولید کامپوننت‌هایی منجر شود که هرگز استفاده نمی‌شوند.

```
Audit Existing UI
      ↓
Define Foundations
      ↓
Create Design Tokens
      ↓
Build Core Components
      ↓
Define Patterns
      ↓
Add Documentation
      ↓
Add Testing
      ↓
Release
      ↓
Collect Feedback
      ↓
Improve
```

۲۹. ممیزی رابط کاربری

UI Audit مرحله بررسی وضع موجود است. هدف آن شناسایی تنوع‌ها، تکرارها و ناسازگاری‌هاست تا تصمیم‌های Design System بر اساس مسئله واقعی گرفته شوند.

- چند نوع Button وجود دارد؟
- چند اندازه Font استفاده شده است؟
- چه رنگ‌های مشابه یا تکراری وجود دارند؟
- چند مدل Input و Form Field داریم؟
- فاصله‌ها چقدر از یکدیگر متفاوت‌اند؟
- کدام Patternها بارها تکرار شده‌اند؟

۳۰. فهرست کامپوننت‌ها

پس از Audit می‌توان فهرستی از Componentهای موجود و موردنیاز تهیه کرد. سپس بر اساس میزان استفاده، اهمیت، ریسک و هزینه نگهداری، آن‌ها را اولویت‌بندی کرد.

```
Button
Input
Select
Checkbox
Radio
Switch
Modal
Tooltip
Card
Tabs
Table
Pagination
Badge
Alert
Toast
Breadcrumb
```

۳۱. آزمون و کنترل کیفیت

Componentهای Design System به دلیل استفاده گسترده، باید سطح مناسبی از تست داشته باشند. خطای کوچک در یک Component پایه ممکن است در تعداد زیادی صفحه تکرار شود.

Unit Test

بررسی منطق داخلی، خروجی و حالات مشخص Component.

Interaction Test

بررسی رفتار کاربر مانند Keyboard، Focus، Click، انتخاب گزینه یا باز شدن Modal.

Visual Regression Test

مقایسه خروجی بصری برای پیدا کردن تغییرات ناخواسته در ظاهر.

Accessibility Test

بررسی قواعد دسترس‌پذیری و تعامل با Keyboard یا Screen Reader.

۳۲. مزایای سیستم طراحی

اثر	مزیت
هماهنگی بصری و رفتاری در کل محصول	Consistency
کاهش نیاز به ساخت مجدد اجزای تکراری	Development Speed
امکان اعمال تغییرات در نقاط مرکزی	Maintainability
رشد کنترل‌شده محصول و تیم	Scalability
ایجاد زبان مشترک میان طراحی و توسعه	Collaboration
استفاده مجدد از اجزای مستند و تست‌شده	Quality

۳۳. چالش‌های سیستم طراحی

- هزینه اولیه برای طراحی و پیاده‌سازی
- نیاز مستمر به Documentation
- مدیریت Breaking Change و Migration
- مقاومت تیم در برابر استانداردها
- Generic شدن بیش از حد Componentها
- اضافه شدن Componentهای غیرضروری
- نیاز به هماهنگی مداوم بین Design و Development

نکته

Design System یک محصول داخلی است و مانند هر محصول دیگری به مالکیت، بازخورد، نسخه‌بندی، نگهداری و تصمیم‌گیری مستمر نیاز دارد.

مرحله 1 - مفاهیم پایه

```
Design System
UI Library
Style Guide
Design Token
Component
Pattern
```

مرحله 2 - Foundations

```
Color
Typography
Spacing
Grid
Radius
Shadow
Icon
Motion
```

مرحله 3 - Component Design

```
Props
Variants
States
Composition
API Design
```

مرحله 4 - Documentation

```
Storybook
Usage Guidelines
Examples
Accessibility Notes
```

مرحله 5 - موضوعات پیشرفته

```
Semantic Tokens
Theming
Dark Mode
Versioning
Testing
Governance
Visual Regression
```

۳۵. ساختار پیشنهادی یک سیستم طراحی

```
design-system
│
├── foundations
│   ├── colors
│   ├── typography
│   ├── spacing
│   ├── radius
│   └── shadows
│
├── tokens
├── components
├── patterns
├── themes
├── icons
├── documentation
└── tests
```

۳۶. ویژگی‌های یک سیستم طراحی بالغ

- Tokenهای استاندارد و چندلایه
- API پایدار و قابل پیش‌بینی
- Documentation کامل و به‌روز
- Accessibility به عنوان بخشی از طراحی
- Testing در چند سطح
- Versioning و Release Process مشخص
- Contribution Guideline
- Governance روشن
- Migration Guide برای تغییرات مهم

۳۷. جمع‌بندی

Design System را می‌توان زیرساخت مشترک تصمیم‌های طراحی و توسعه رابط کاربری دانست. ارزش آن زمانی مشخص می‌شود که تیم مجبور نباشد برای هر صفحه و هر Feature دوباره درباره همان رنگ، فاصله، رفتار Button یا الگوی نمایش خطا تصمیم‌گیری کند.

```
Foundations
+
Design Tokens
+
Components
+
Patterns
+
Documentation
+
Accessibility
+
Testing
+
Governance
```

یک Design System موفق باید برای Designer قابل فهم، برای Developer قابل استفاده و برای محصول قابل توسعه باشد.
هدف نهایی، ایجاد محدودیت بی‌دلیل نیست؛ بلکه تبدیل تصمیم‌های تکراری به استانداردهای واضح و قابل اعتماد است.